■ EcoScan PK

The AI Munshi for Pakistan's Contractors

"Point. Scan. Buy Smarter."

ITU Startup Competition 2026	4 Members	4 Hours	$100 Credits

What this document is

This is a complete, step-by-step build guide for all four team members. Every section tells you exactly **which website to open**, **what to click**, **what code to paste**, **which terminal command to run**, and **which AI tool to prompt**. Nothing is assumed. Follow each step in order and you will have a working demo in 4 hours.

Tech Stack at a Glance

Service	Purpose	Cost
Gemini 1.5 Flash (Google AI Studio)	Classifies material from photo	~$0.50 total
Google Cloud Text-to-Speech (ur-PK)	Speaks Urdu response aloud	FREE (1M chars/month)
Firebase Hosting	Deploys your web app live	FREE
Firebase Firestore	Stores scan history	FREE
FastAPI + Python	Backend server (runs locally)	FREE
WhatsApp wa.me link	Supplier contact button	FREE — no API key needed

Timeline at a Glance

Hour	M1 — Cloud & APIs	M2 — Frontend UI	M3 — Backend Logic	M4 — Pitch & Demo
0–1	GCP project, enable APIs, Gemini key, test in AI Studio	Open VS Code, scaffold React app with Vite	Test Gemini prompt in AI Studio, install Python deps	Start pitch deck from template
1–2	Firebase Hosting config, GitHub repo, share all keys	Paste full App.jsx, wire keys, run dev server	Write /scan and /speak FastAPI endpoints	Firebase console setup, Firestore rules
2–3	Support teammates with any config/auth errors	History table UI, mobile styling polish	analyze() function, Firestore write logging	Write demo script, 3 rehearsal scenarios
3–4	First firebase deploy attempt	Final UI polish + loading states	End-to-end debug on real photos	Mobile device testing
4	Redeploy if needed	Join M3 for final integration test	Final integration test	Q&A prep, record backup video
LAST 30 MIN	**ALL FOUR MEMBERS — full rehearsal x2 with real photos on stage phone**			

AI Tools Summary — What Each One Is For

Tool	URL	What to use it for
Google AI Studio	aistudio.google.com	Get Gemini API key + test your prompt before writing code
Google Cloud Console	console.cloud.google.com	Enable APIs, create service accounts, billing

Firebase Console	console.firebase.google.com	Firestore database + Hosting deployment
Vite (CLI tool)	run: npm create vite@latest	Scaffold React app in seconds
Claude (this AI)	claude.ai	Fix broken JS/Python, explain errors, generate missing code
Gamma	gamma.app	Generate pitch deck slides from a text prompt

The Golden Rule: No one writes CSS from scratch or debugs blindly. Every styling problem → Claude. Every broken function → Claude with the exact error pasted in.

MEMBER 1 Cloud & APIs

Gets everyone unblocked. Owns all keys and config. Speed = team success.

Total time commitment: 1.5 hrs setup + support role for remaining time

Your job is to get everyone else unblocked. You own all API keys and cloud config. Once you finish Hour 1, the other three members can start their real work. Speed is everything — work through every step without stopping.

1 Create Google Cloud Project

■ 5 min | ■ Browser → console.cloud.google.com

This is the master container for everything. One project for the whole team — do not create separate ones.

- Open Chrome on your laptop.
- Go to this exact URL: console.cloud.google.com
- Sign in with the team Google account (the one you all agreed on).
- At the top of the screen, click the project dropdown button — it usually says "My First Project" or similar.
- Click "NEW PROJECT".
- In the Project name field type exactly: ecoscan-pk
- Leave Location as "No organization" unless your account is a university Workspace account.
- Click "CREATE". Wait about 30 seconds.
- The dropdown at the top should now show "ecoscan-pk". If not, click the dropdown and select it.

2 Enable Billing (Critical — do this before anything else)

■ 5 min | ■ Google Cloud Console → Billing

Text-to-Speech and Speech-to-Text will silently fail without billing, even though the free tier covers everything. This is the #1 mistake teams make.

- In the left sidebar click the three-line menu (■) → "Billing".
- If you have $25 Google credits: click "Redeem a promotion or coupon" and enter the code.
- If no coupon: click "Link a billing account" → add a debit or credit card. You will NOT be charged within free limits.
- Click the menu again → hover "Billing" → click "Overview" — confirm the project "ecoscan-pk" is linked to a billing account.

■■ Do NOT skip this. APIs will appear enabled but return 403 errors without billing.

3 Enable the 3 Required APIs

■ 5 min | ■ Google Cloud Console → APIs & Services → Library

Each API needs to be individually switched on.

- Left sidebar → "APIs & Services" → "Library".
- In the search box type "Generative Language API" → click the result → click "ENABLE". Wait for green tick.
- Click the back arrow. Search "Cloud Text-to-Speech API" → click result → "ENABLE".
- Click back arrow. Search "Cloud Speech-to-Text API" → click result → "ENABLE".
- All three should now appear under APIs & Services → Enabled APIs.

4 Get Your Gemini API Key

■ 3 min | ■ Browser → aistudio.google.com

This key goes into the backend Python code. Share it with your team immediately.

- Open a new tab. Go to: aistudio.google.com
- Sign in with the same Google account.
- In the left sidebar click "Get API key".
- Click "Create API key in new project" — BUT choose the existing project "ecoscan-pk" from the dropdown.
- Click "Create API key".
- Copy the full key (starts with "AIza..."). It is very long.
- Paste it into the team WhatsApp group chat immediately. Label it: GEMINI_KEY=

■■ This key is secret. Do not commit it to GitHub. Do not share it publicly.

5 Set Up Firebase Project

■ 8 min | ■ Browser → console.firebase.google.com

Firebase gives you free hosting and a real database. Link it to the same Google Cloud project.

- Open new tab → go to: console.firebase.google.com
- Click "Add project".
- You will see "Add Firebase to a Google Cloud project" — click that option.
- In the dropdown select "ecoscan-pk".
- Click Continue. Disable Google Analytics (not needed). Click "Add Firebase".
- Wait for project to be created (about 1 minute).
- Left sidebar → Build → "Firestore Database" → "Create database".
- Select "Start in test mode" → click Next.
- Choose location "nam5 (us-central)" → click "Enable".
- Now left sidebar → Build → "Hosting" → "Get started" → click Next on all screens until you see deployment instructions. Stop here — do not run commands yet.

6 Get Firebase Config Object

■ 5 min | ■ Firebase Console → Project Settings

This JSON blob goes into the frontend React code. Share it immediately.

- In Firebase Console, click the gear icon (■) next to "Project Overview" in the top left.
- Click "Project settings".
- Scroll down to "Your apps" section.
- Click the web icon (looks like:).
- Register app name: ecoscan-web → click "Register app".
- You will see a code block starting with "const firebaseConfig = {". Copy the ENTIRE block.
- Paste into team WhatsApp immediately. Label it: FIREBASE_CONFIG

```
// It will look like this — yours will have real values: const firebaseConfig = { apiKey:
"AIzaSyXXXXXXXX", authDomain: "ecoscan-pk.firebaseapp.com", projectId: "ecoscan-pk",
storageBucket: "ecoscan-pk.appspot.com", messagingSenderId: "1234567890", appId:
"1:1234567890:web:abcdef123456" };
```

7 Install Node.js and Firebase CLI

■ 8 min | ■ nodejs.org download + terminal/command prompt

The Firebase CLI is a command-line tool that lets you deploy from your laptop.

- Open your browser → go to: nodejs.org
- Download the LTS version (left button). Install it — click through all the defaults.
- Now open a terminal:
- • Windows: Press Windows key → type "cmd" → press Enter to open Command Prompt.
- • Mac: Press Cmd+Space → type "terminal" → press Enter.
- Type this command and press Enter: node --version
- You should see something like v20.11.0. If you see an error, restart your computer and try again.
- Now type: npm install -g firebase-tools and press Enter. Wait for it to finish.
- Now type: firebase login and press Enter.
- A browser window will open — log in with the team Google account.
- You should see "Firebase CLI Login Successful". Go back to the terminal.

■ *If npm install fails with a permissions error on Mac, prefix the command with: sudo npm install -g firebase-tools*

8 Create Project Folder and Init Firebase

■ 7 min | ■ Terminal / Command Prompt

This creates the actual project folder that Member 2 will put code into.

- In your terminal, decide where to create the project folder (Desktop is fine).
- On Windows type: cd Desktop
- On Mac type: cd ~/Desktop
- Then type: mkdir ecoscan-pk and press Enter.
- Then type: cd ecoscan-pk and press Enter. You are now inside the project folder.
- Type: firebase init and press Enter.
- Use arrow keys to navigate. Press SPACEBAR to select "Hosting" then "Firestore".
- Press Enter to continue.
- Choose "Use an existing project" → select "ecoscan-pk".
- For public directory type a dot (.) and press Enter.
- Configure as single-page app? Type: N and press Enter.
- Set up automatic builds? Type: N and press Enter.
- Firebase init complete! You should see a "Firebase initialization complete!" message.
- Share the full folder path with Member 2. On Windows right-click the folder → Properties → Location.

9 Create GitHub Repo and Share Access

■ 5 min | ■ Browser → github.com

Everyone will push code to this shared repository.

- Go to: github.com → Sign in.
- Click the + icon (top right) → "New repository".
- Repository name: ecoscan-pk
- Set to Public.
- Check "Add a README file".
- Click "Create repository".
- Click "Settings" tab → "Collaborators" → "Add people".
- Add all 3 teammates by their GitHub username.
- Share the repo URL in team chat: https://github.com/YOURNAME/ecoscan-pk

■ HOUR 1 CHECKPOINT

Done?	Item
■	Gemini API key shared in team chat
■	Firebase config object shared in team chat
■	Firebase Hosting and Firestore enabled
■	firebase-tools CLI installed and logged in
■	Project folder created with firebase init
■	GitHub repo created, teammates added

Hours 2–4: Support Role

Once the setup is done, your role shifts to team unblocking. Sit next to Member 3 (backend). Watch for these common errors and fix them immediately:

Error Message	Fix
403 PERMISSION_DENIED	Billing not enabled — go back to Step 2
GOOGLE_APPLICATION_CREDENTIALS not set	See Member 3 Step 2 — set the env variable in terminal
firebase: command not found	npm install -g firebase-tools was not run, or terminal needs restart
CORS error in browser console	FastAPI CORSMiddleware not added — Member 3 must paste the middleware code
ModuleNotFoundError: No module named fastapi	pip install fastapi uvicorn google-generativeai was not run

Hour 3–4: Deploy to Firebase Hosting

Once Member 2 runs: npm run build — you take over deployment:

```
# In terminal, navigate to the project folder: cd Desktop/ecoscan-pk # Copy the built React files: # Member 2 will have a "dist" folder — copy everything from it to your project folder # Deploy: firebase deploy --only hosting # You will see a live URL like: # Hosting URL: https://ecoscan-pk.web.app # Share this URL with everyone immediately!
```

MEMBER 2 Frontend UI

Owns everything the judges see. Mobile-first React app with camera scan + Urdu voice + WhatsApp.

Total time commitment: 4 hrs active — building from Hour 0

You own everything the judges see and touch. Your job is a beautiful, working mobile-first React app that connects to the backend. Wait for Member 1 to share the Firebase config and Gemini key before starting Hour 2.

1 Install VS Code

■ 5 min | ■ Browser → code.visualstudio.com

VS Code is the code editor you will use to write and edit the frontend.

- Open browser → go to: code.visualstudio.com
- Click the big download button for your OS (Windows or Mac).
- Install it — click through all defaults.
- Open VS Code. You should see a welcome screen.

2 Install Node.js (if Member 1 has not already done it on this laptop)

■ 5 min | ■ Browser → nodejs.org

Node.js lets you run the React build tools.

- Go to: nodejs.org → download LTS version → install.
- Open terminal (Ctrl+` in VS Code opens terminal inside VS Code — use this!)
- Type: node --version → should show v18 or v20.

3 Scaffold React App with Vite

■ 5 min | ■ VS Code terminal

Vite sets up a complete React project structure in seconds.

- In VS Code, press Ctrl+` (backtick) to open the built-in terminal.
- Navigate to your Desktop: type cd Desktop and press Enter.
- Run this command exactly:

```
npm create vite@latest ecoscan-frontend -- --template react
```

- When prompted to install packages: press Y and Enter.
- When asked project name: ecoscan-frontend
- Framework: React. Variant: JavaScript.
- Then run these three commands one by one:

```
cd ecoscan-frontend npm install npm install firebase
```

- Finally start the development server:

```
npm run dev
```

- Open browser → go to: http://localhost:5173 — you should see the Vite + React welcome page.

4 Open the Project in VS Code

■ 2 min | ■ VS Code

- In VS Code: File → Open Folder → navigate to Desktop → ecoscan-frontend → click Open.
- In the left sidebar (Explorer panel) you will see the folder structure.
- The main file you will edit is: src/App.jsx
- Open it by clicking on it in the sidebar.

5 Replace App.jsx with Full EcoScan Frontend

■ 15 min | ■ VS Code → src/App.jsx

Delete everything in App.jsx and paste this complete code. Replace YOUR_GEMINI_KEY and YOUR_FIREBASE_CONFIG with the values Member 1 shared.

- Click on src/App.jsx in the left sidebar to open it.
- Press Ctrl+A to select all the existing code.
- Press Delete to clear it.
- Paste the entire code block below.
- Find the line that says YOUR_BACKEND_URL and replace with: http://localhost:8000
- Save the file: Ctrl+S.

```
import { useState, useRef } from "react"; import { initializeApp } from "firebase/app";
import { getFirestore, addDoc, collection, getDocs, query, orderBy, limit } from
"firebase/firestore"; // PASTE YOUR FIREBASE CONFIG FROM MEMBER 1 HERE: const firebaseConfig
= { apiKey: "REPLACE_WITH_FIREBASE_KEY", authDomain: "ecoscan-pk.firebaseapp.com",
projectId: "ecoscan-pk", storageBucket: "ecoscan-pk.appspot.com", messagingSenderId:
"REPLACE_WITH_SENDER_ID", appId: "REPLACE_WITH_APP_ID" }; const firebaseApp =
initializeApp(firebaseConfig); const db = getFirestore(firebaseApp); const BACKEND =
"http://localhost:8000"; export default function App() { const [result, setResult] =
useState(null); const [loading, setLoading] = useState(false); const [history, setHistory] =
useState([]); const [error, setError] = useState(""); const fileRef = useRef(); const
scanImage = async (imageFile) => { setLoading(true); setError(""); try { const formData =
new FormData(); formData.append("file", imageFile); const res = await
fetch(`${BACKEND}/scan`, { method: "POST", body: formData }); if (!res.ok) throw new
Error("Backend error: " + res.status); const data = await res.json(); setResult(data);
speakUrdu(data.urdu_response); await logScan(data); } catch (e) { setError("Error: " +
e.message + " — Is the backend running? See Member 3."); } setLoading(false); }; const
speakUrdu = async (text) => { try { const res = await fetch(`${BACKEND}/speak`, { method:
"POST", headers: { "Content-Type": "application/json" }, body: JSON.stringify({ text }) });
const data = await res.json(); const audio = new Audio("data:audio/mp3;base64," +
data.audio); audio.play(); } catch (e) { console.error("TTS error:", e); } }; const logScan
= async (data) => { try { await addDoc(collection(db, "scans"), { material: data.material,
timestamp: new Date(), city: "Lahore", saving: data.saving }); } catch (e) {
console.error("Firestore error:", e); } }; const openWhatsApp = (material) => { const msg =
`Assalam o Alaikum, mujhe ${material} chahiye. Aapka current rate kya hai? (EcoScan PK se
bheja)`; window.open(`https://wa.me/923001234567?text=${encodeURIComponent(msg)}`); }; const
handleFile = (e) => { const file = e.target.files[0]; if (file) scanImage(file); }; return (
<div style={{ maxWidth: 430, margin: "0 auto", fontFamily: "sans-serif", background:
"#f0fdf4", minHeight: "100vh", padding: "16px" }}> <div style={{ textAlign: "center",
marginBottom: 20 }}> <div style={{ fontSize: 40 }}>■</div> <h1 style={{ color: "#1B6B3A",
margin: 0, fontSize: 26 }}>EcoScan PK</h1> <p style={{ color: "#555", margin: "4px 0 0"
}}>AI Munshi for Pakistan Contractors</p> </div> <div onClick={() =>
fileRef.current.click()} style={{ background: "#1B6B3A", borderRadius: 16, padding: "32px
16px", textAlign: "center", cursor: "pointer", color: "white", boxShadow: "0 4px 12px
rgba(27,107,58,0.3)" }}> <div style={{ fontSize: 48 }}>■</div> <p style={{ margin: 0,
fontSize: 18, fontWeight: "bold" }}> {loading ? "Scanning... ■" : "Tap to Scan Material"}
</p> <p style={{ margin: "6px 0 0", fontSize: 12, opacity: 0.8 }}> Point camera at brick,
```

```
concrete, steel, wood, or tile </p> </div> <input ref={fileRef} type="file" accept="image/*"
capture="environment" style={{ display: "none" }} onChange={handleFile} /> {error && <div
style={{ background:"#fff3cd", border:"1px solid #ff6f00", borderRadius:8, padding:10,
marginTop:12, color:"#BF360C", fontSize:13 }}>{error}</div>} {result && ( <div style={{
background: "white", borderRadius: 16, padding: 20, marginTop: 16, boxShadow: "0 2px 8px
rgba(0,0,0,0.08)" }}> <h2 style={{ margin: "0 0 4px", color: "#1B6B3A"
}}>{result.material}</h2> <p style={{ margin: "0 0 12px", color: "#666", fontSize: 13 }}>
Confidence: {Math.round((result.confidence || 0.9) * 100)}% </p> <div style={{ display:
"grid", gridTemplateColumns: "1fr 1fr", gap: 10 }}> <div style={{ background:"#f0fdf4",
borderRadius:10, padding:10 }}> <div style={{ fontSize:11, color:"#666" }}>Carbon
(CO2/kg)</div> <div style={{ fontSize:20, fontWeight:"bold", color:"#1B6B3A" }}>
{result.carbon} kg </div> </div> <div style={{ background:"#fff8e1", borderRadius:10,
padding:10 }}> <div style={{ fontSize:11, color:"#666" }}>Market Price</div> <div style={{
fontSize:20, fontWeight:"bold", color:"#FF6F00" }}> Rs. {result.cost} </div> </div> </div>
<div style={{ background:"#e8f5e9", borderRadius:12, padding:14, marginTop:12 }}> <div
style={{ fontSize:11, color:"#555", marginBottom:4 }}>■■ Greener Alternative</div> <div
style={{ fontWeight:"bold", fontSize:16, color:"#1B6B3A" }}>{result.alt}</div> <div style={{
color:"#2E7D32", fontSize:13, marginTop:4 }}> {result.saving}% less carbon · Rs.
{result.alt_carbon} CO2/kg </div> </div> <div style={{ background:"#f3e5f5",
borderRadius:12, padding:14, marginTop:10 }}> <div style={{ fontSize:11, color:"#666",
marginBottom:4 }}>■■ Urdu AI Response</div> <div style={{ fontSize:13, color:"#4A148C",
fontStyle:"italic" }}> {result.urdu_response} </div> </div> <button onClick={() =>
openWhatsApp(result.material)} style={{ width:"100%", marginTop:14, background:"#25D366",
color:"white", border:"none", borderRadius:12, padding:"14px 0", fontSize:16,
fontWeight:"bold", cursor:"pointer" }}> ■ Contact Supplier on WhatsApp </button> </div> )}
<div style={{ marginTop: 20, background:"white", borderRadius:12, padding:14, boxShadow:"0
2px 8px rgba(0,0,0,0.05)" }}> <h3 style={{ margin:"0 0 10px", color:"#333", fontSize:14 }}>■
Recent Scans</h3> {history.length === 0 && <p style={{ color:"#999", fontSize:12, margin:0
}}> No scans yet. Scan your first material above!</p>} {history.map((h, i) => ( <div key={i}
style={{ display:"flex", justifyContent:"space-between", padding:"6px 0", borderBottom:"1px
solid #f0f0f0", fontSize:13 }}> <span>{h.material}</span> <span style={{ color:"#1B6B3A",
fontWeight:"bold" }}>{h.saving}% saved</span> </div> ))} </div> </div> ); }
```

6 Test the Frontend (Before Backend is Ready)

■ 5 min | ■ Browser → localhost:5173

Make sure the UI renders correctly even before the backend is connected.

- The dev server should already be running (from Step 3).
- Open browser → go to: http://localhost:5173
- You should see the green EcoScan UI with a camera scan button.
- Try tapping the scan button — it will open a file picker (or camera on phone).
- If you see an error message in the app, that is OK — it means the backend is not running yet. That is Member 3's job.
- Check that the layout looks good on a narrow window (simulate phone: press F12 → click phone icon in Chrome devtools).

7 Build for Deployment

■ 3 min | ■ VS Code terminal

Once the backend works end-to-end, create the production build.

- In the terminal (Ctrl+` in VS Code):
- Run: npm run build
- This creates a "dist" folder with optimized files.
- Tell Member 1 the build is done — they will deploy it to Firebase Hosting.

Styling Tips — If the UI looks broken

Problem	What to type in claude.ai
Button too small on phone	"Make this React button larger, min-height 56px, full width on mobile"
Text overflowing container	"Fix this CSS overflow issue in React" + paste the component code
Layout not centered	"Center this React component vertically and horizontally on mobile"
Colors look wrong	"Change the primary color to #1B6B3A in this React component"

MEMBER 3 Backend Logic

Owns Gemini Vision + Urdu TTS + FastAPI server. The brain of the whole app.

Total time commitment: 4 hrs active — start with AI Studio testing immediately

You own the brain of the app — the FastAPI server that calls Gemini Vision and Google TTS. Start Hour 0 testing your Gemini prompt in AI Studio. By Hour 1, paste the full backend code. By Hour 2, Urdu audio should be playing.

1 Test Gemini Prompt in AI Studio (Hour 0 — Start Here)

■ 20 min | ■ Browser → aistudio.google.com

Before writing any code, validate that Gemini can identify building materials. This saves debugging time later.

- Go to: aistudio.google.com
- Sign in with the team Google account.
- Click "Create new prompt" or "Try Gemini" in the center.
- Make sure model is set to "Gemini 1.5 Flash" (dropdown at top).
- In the prompt text box, paste this exact prompt:
- --- AFTER PASTING PROMPT ---
- Click the image icon (■) in the prompt area to upload a test image.
- Use a photo of a brick, concrete wall, or any building material.
- Click "Run".
- The response should be pure JSON like: {"material": "Fired Brick", "confidence": 0.92}
- If it works: your prompt is ready. Save it.
- If it returns text around the JSON: add to your prompt: "Return JSON only. No preamble. No explanation."

```
Identify the building material shown in this image. Reply ONLY with JSON — no other text, no explanation: {"material": "Fired Brick", "confidence": 0.91} Choose material from ONLY these options: Fired Brick, Concrete, Steel Rebar, Timber, Ceramic Tile If unsure, choose the closest match.
```

2 Set Up Python Environment

■ 10 min | ■ Python installer + terminal

Install Python and all required libraries.

- Check if Python is installed: open terminal → type: python --version
- If you see Python 3.10 or higher → skip to the pip install step.
- If not installed: go to python.org → download Python 3.12 → install (check "Add to PATH").
- Open a new terminal after installing.
- Run this command to install all dependencies:
- Wait for all packages to install (may take 2-3 minutes).
- If you get a "pip not found" error: try python -m pip install ... instead.

```
pip install fastapi uvicorn google-generativeai google-cloud-texttospeech Pillow python-multipart
```

3 Create the Backend File

■ 5 min | ■ VS Code or any text editor

Create a new file called main.py in the same folder as the React app.

- Open VS Code.
- File → Open Folder → choose the ecoscan-pk folder (from Member 1).
- In the left sidebar, right-click → New File.
- Name it: main.py
- Open it by clicking on it.

4 Paste the Complete Backend Code

■ 10 min | ■ VS Code → main.py

Paste this entire code block into main.py. Replace YOUR_GEMINI_KEY with the key Member 1 shared.

- Select all existing content in main.py (Ctrl+A) and delete.
- Paste the full code below.
- Find the line: genai.configure(api_key="YOUR_GEMINI_KEY")
- Replace YOUR_GEMINI_KEY with the actual key from Member 1's WhatsApp message.
- Save: Ctrl+S.

```
from fastapi import FastAPI, UploadFile, File from fastapi.middleware.cors import
CORSMiddleware import google.generativeai as genai from google.cloud import texttospeech
from PIL import Image import io, json, base64 app = FastAPI() # Allow React frontend to call
this server app.add_middleware( CORSMiddleware, allow_origins=["*"], allow_credentials=True,
allow_methods=["*"], allow_headers=["*"], ) # Configure Gemini — replace with real key from
Member 1 genai.configure(api_key="YOUR_GEMINI_KEY") model =
genai.GenerativeModel("gemini-1.5-flash") # Materials database with eco alternatives (Lahore
market prices) MATERIALS = { "Fired Brick": {"carbon":0.24,"cost":12, "alt":"AAC Blocks",
"alt_carbon":0.09,"saving":62,"urdu":"fired brick"}, "Concrete":
{"carbon":0.41,"cost":180,"alt":"Fly Ash Concrete",
"alt_carbon":0.21,"saving":49,"urdu":"concrete"}, "Steel Rebar":
{"carbon":1.46,"cost":320,"alt":"Recycled Steel",
"alt_carbon":0.51,"saving":65,"urdu":"steel saria"}, "Timber":
{"carbon":0.31,"cost":850,"alt":"Bamboo", "alt_carbon":0.05,"saving":84,"urdu":"lakri"},
"Ceramic Tile": {"carbon":0.59,"cost":95, "alt":"Recycled Glass Tile",
"alt_carbon":0.22,"saving":63,"urdu":"ceramic tile"}, } @app.get("/health") def health():
return {"status": "running", "message": "EcoScan PK backend is alive"} @app.post("/scan")
async def scan_material(file: UploadFile = File(...)): img_bytes = await file.read() image =
Image.open(io.BytesIO(img_bytes)) prompt = """Identify the building material shown in the
image. Reply ONLY with JSON — no other text: {"material": "Fired Brick", "confidence": 0.91}
Choose from ONLY: Fired Brick, Concrete, Steel Rebar, Timber, Ceramic Tile""" response =
model.generate_content([prompt, image]) text = response.text.strip() # Clean up response if
Gemini adds backticks if text.startswith("```"): text = text.split("```")[1] if
text.startswith("json"): text = text[4:] text = text.strip() result = json.loads(text)
material_name = result.get("material", "Concrete") d = MATERIALS.get(material_name,
MATERIALS["Concrete"]) urdu_text = ( f"Yeh {d['urdu']} hai. " f"Eco alternative {d['alt']}
hai " f"jo {d['saving']} fisad kam carbon deta hai " f"aur environment ke liye behtar hai."
) return { "material": material_name, "confidence": result.get("confidence", 0.9), "carbon":
d["carbon"], "cost": d["cost"], "alt": d["alt"], "alt_carbon": d["alt_carbon"], "saving":
d["saving"], "urdu_response": urdu_text, } @app.post("/speak") async def speak_urdu(data:
dict): client = texttospeech.TextToSpeechClient() synthesis_input =
texttospeech.SynthesisInput(text=data["text"]) voice = texttospeech.VoiceSelectionParams(
language_code="ur-PK", ssml_gender=texttospeech.SsmlVoiceGender.NEUTRAL ) audio_config =
texttospeech.AudioConfig( audio_encoding=texttospeech.AudioEncoding.MP3 ) response =
client.synthesize_speech( input=synthesis_input, voice=voice, audio_config=audio_config )
audio_b64 = base64.b64encode(response.audio_content).decode("utf-8") return {"audio":
audio_b64}
```

5 Set Up Google Cloud Credentials for TTS

■ 10 min | ■ Google Cloud Console → terminal

The Text-to-Speech API needs a service account key file to authenticate.

- Go to: console.cloud.google.com → make sure project "ecoscan-pk" is selected.
- Left sidebar → "APIs & Services" → "Credentials".
- Click "Create Credentials" → "Service account".
- Service account name: ecoscan-tts
- Click "Create and Continue".
- Role: click the dropdown → search "Cloud Text-to-Speech" → select "Cloud Text-to-Speech Client".
- Click "Continue" → "Done".
- Back on the Credentials page, click on the service account you just created.
- Click "Keys" tab → "Add Key" → "Create new key" → JSON → "Create".
- A JSON file downloads automatically. Note where it saves (usually Downloads folder).
- In your terminal, set the environment variable:

```
# Windows Command Prompt: set
GOOGLE_APPLICATION_CREDENTIALS=C:\Users\YourName\Downloads\ecoscan-tts-key.json # Mac/Linux
terminal: export
GOOGLE_APPLICATION_CREDENTIALS="/Users/YourName/Downloads/ecoscan-tts-key.json" # IMPORTANT:
Replace the path with the actual path to your downloaded JSON file
```

■■ You must set this environment variable in the SAME terminal window you will run the server from. If you close and reopen terminal, you must set it again.

6 Run the Backend Server

■ 3 min | ■ Terminal (same window where you set GOOGLE_APPLICATION_CREDENTIALS)

Start the FastAPI server so the frontend can connect to it.

- Make sure you are in the ecoscan-pk folder: type ls (Mac) or dir (Windows) — you should see main.py.
- Run: uvicorn main:app --reload
- You should see: "Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)"
- Open browser → go to: http://localhost:8000/health
- You should see: {"status":"running","message":"EcoScan PK backend is alive"}
- If you see this → backend is working. Tell Member 2 to test the scan.

```
uvicorn main:app --reload
```

7 Test the /scan Endpoint

■ 5 min | ■ Browser → localhost:8000/docs

Manually test that Gemini can identify a material via the API.

- Go to: http://localhost:8000/docs (FastAPI auto-generates a test UI)
- Click on POST /scan → "Try it out".
- Click "Choose File" → select any photo of a brick or concrete wall.
- Click "Execute".
- Scroll down to see the response — it should include material, carbon, cost, urdu_response.
- If you see an error → paste the exact error message into claude.ai for help.

8 Test the /speak Endpoint (Urdu TTS)

■ 5 min | ■ Browser → localhost:8000/docs

Verify that Urdu audio is generated.

- In the FastAPI docs page: click POST /speak → "Try it out".
- In the request body replace the default with the JSON below.
- Click "Execute".
- You should see a response with a long "audio" field (base64 string).
- If you see this → TTS is working.
- The audio will actually play when Member 2's frontend calls this endpoint.

```
{"text": "Yeh fired brick hai. AAC blocks 62 fisad kam carbon dete hain."}
```

Common Backend Errors & Fixes

	Fix
NDecodeError	Gemini returned text around the JSON. The code already handles this — if still failing, add more cleanup: text = text.replace("json"
th.exceptions.DefaultCredentialsError	GOOGLE_APPLICATION_CREDENTIALS env variable not set. Redo Step 5.
on refused (from frontend)	uvicorn server not running. Run: uvicorn main:app --reload
rror: NoneType	Material not found in MATERIALS dict. Add a default fallback: MATERIALS.get(name, MATERIALS["Concrete"])
mat error	Add try/except around Image.open() and return a 400 error

MEMBER 4 Pitch & Demo

The voice of the team. Owns the slides, the script, the demo, and the Q&A.;

Total time commitment: 4 hrs active — start Gamma immediately in Hour 0

You are the voice of the team. Your job is to build a killer 5-minute pitch, write a rehearsed demo script, set up Firestore from the console, and make sure the team is ready to perform on stage. Most competing teams will have slides only — you will have a working product.

1 Create Pitch Deck with Gamma (Hour 0)

■ 20 min | ■ Browser → gamma.app

Gamma AI generates a full slide deck from a text prompt. Use it — do not build slides from scratch.

- Open browser → go to: gamma.app
- Click "Sign up" or "Log in" → use Google account.
- Click "Create new" → choose "Presentation".
- Choose "Generate with AI".
- Paste the prompt below into the text box.
- Click "Generate". Wait 30 seconds.
- Review the slides — edit any text that does not match.
- Click the image icon on Slide 4 and upload a screenshot of the app once Member 2 has it running.
- Export as PDF for backup: File → Export → PDF.

```
Create a 7-slide pitch deck for EcoScan PK — an AI app for Pakistani construction
contractors. Slide 1 — Title: EcoScan PK | The AI Munshi for Pakistan Contractors | ITU
Startup 2026 Slide 2 — Problem: 2 million small contractors in Pakistan. Zero digital tools.
Buy on price alone. No one measures carbon or savings. Slide 3 — Solution: Point phone at
material. Gemini AI identifies it. App speaks result in Urdu. WhatsApp connects to supplier.
Slide 4 — Live Demo Screenshot placeholder Slide 5 — Tech Stack: Gemini 1.5 Flash + Google
TTS (Urdu) + Firebase + FastAPI. Built in 4 hours on $5 of credits. Slide 6 — Business
Model: Free for contractors. Rs. 50/lead from suppliers. Rs. 5,000/month verified listings.
Rs. 75,000/month NGO contracts. Rs. 325,000/month Year 1 target. Slide 7 — Ask: Google Cloud
credits + mentor to reach 1,000 Lahore contractors by Q3 2026. Color theme: deep green
(#1B6B3A) and white. Clean, modern, professional.
```

2 Set Up Firestore Database via Console (Hour 1)

■ 15 min | ■ Browser → console.firebase.google.com

Add all 5 material records to Firestore manually so the app has real data from the start.

- Go to: console.firebase.google.com → select project "ecoscan-pk".
- Left sidebar → Build → Firestore Database.
- Click "Start collection".
- Collection ID: materials → click Next.
- Document ID: Fired Brick
- Add the fields shown in the code block below (click "+ Add field" for each).
- Click "Save" to save Fired Brick.
- Click "+ Add document" to add the next material.

• Add all 5 materials using the data table that follows.

```
// Fired Brick document fields: carbon (number) → 0.24 cost (number) → 12 alt (string) → AAC Blocks alt_carbon(number) → 0.09 saving (number) → 62 urdu (string) → fired brick unit (string) → per brick // Repeat for each material below using the table on this page
```

All 5 Materials to Add to Firestore

Document ID	carbon	cost	alt	saving	unit
Fired Brick	0.24	12	AAC Blocks	62%	per brick
Concrete	0.41	180	Fly Ash Concrete	49%	per bag
Steel Rebar	1.46	320	Recycled Steel	65%	per kg
Timber	0.31	850	Bamboo	84%	per cft
Ceramic Tile	0.59	95	Recycled Glass Tile	63%	per tile

3 Write the Demo Script

■ 20 min | ■ Paper or Notes app

Memorise this script. The demo takes exactly 2 minutes and is the strongest part of the pitch.

```
DEMO SCRIPT — 2 minutes — memorise this: [Hold up your phone with a photo of a brick on screen] "Let me show you EcoScan PK in action." [Tap the scan button] "I am scanning this fired brick right now — this is a live Gemini API call." [Wait 3 seconds for result to appear] "Gemini has identified it as a Fired Brick." [The Urdu voice plays automatically — do not speak over it] [After the voice: point at the screen] "The app just said in Urdu: Yeh fired brick hai. AAC blocks 62 fisad kam carbon dete hain." Meaning: This is a fired brick. AAC blocks give 62% less carbon. [Point at the green alternative section] "Our AI instantly recommended a greener alternative — AAC Blocks — with 62% lower carbon emissions and only 8% higher cost." [Tap the WhatsApp button] "And with one tap, the contractor can contact a supplier directly. No English needed. No computer needed. Just a phone." [Pause — let it land] "We built this in 4 hours. On $5 of Google Cloud credits."
```

4 Prepare 3 Backup Demo Scenarios

■ 15 min | ■ Your phone camera

Have 3 different material photos ready in your phone camera roll as backups.

- Scenario 1 (primary): Fired Brick — take a photo of any brick wall.
- Scenario 2 (backup): Concrete — photo of a grey concrete surface.
- Scenario 3 (backup 2): Timber/wood — photo of any wood surface.
- Save all 3 photos in a folder called "EcoScan Demo" in your phone gallery.
- Test each photo through the app once it is working (Hour 3).
- If the app crashes on stage → have the demo video (see Step 6) ready.

5 Prepare Q&A; Answers

■ 10 min | ■ Notes app or paper

Judges always ask these questions. Prepare short, confident answers.

```
LIKELY JUDGE QUESTIONS — memorise these answers: Q: "How is this different from a Google Image Search?" A: "Google tells you what something is. We tell you what it costs, how much carbon it produces, and what to buy instead — all in Urdu, spoken aloud, with a WhatsApp supplier link." Q: "How do you make money?" A: "Day 1: free for contractors, Rs. 50 per qualified supplier lead. Month 6: Rs. 5,000/month verified supplier listings. Year 2: Rs. 75,000/month government and NGO API contracts. Year 1 target: Rs. 325,000/month." Q: "How do
```

```
you get contractors to use it?" A: "WhatsApp groups for construction workers in Lahore.
Supplier partnerships — they promote it to their buyers. Free forever for contractors — zero
friction." Q: "What if Gemini misidentifies a material?" A: "Confidence score is shown.
Below 70% we show a 'not sure' message and ask the user to retake the photo. Accuracy
improves as we fine-tune on Pakistani construction images." Q: "Can you scale beyond
Lahore?" A: "The same codebase works for any city. We add a city selector and update the
price database per city. Karachi, Islamabad — 2 hours of database work."
```

6 Record a Backup Demo Video Tonight

■ 10 min | ■ Your phone

WiFi fails at competitions. A recorded video saves you.

- Tonight, when the app is fully working, do a complete demo run.
- Screen record on your phone: Android: swipe down → Screen Record. iPhone: Control Centre → Screen Recording.
- Go through the full demo script above.
- Save the video. Keep it in your phone home screen for instant access.
- Trim it to exactly 90 seconds using the phone Photos app.

■■ If WiFi fails on stage, say: "In case of connectivity issues, here is a recorded demo from our testing session." Play the video. Judges accept this.

5-Minute Pitch Timing Breakdown

gment	Time	Key Line to Memorise
ok	30 sec	"Pakistan produces 100 million bricks a year. Every contractor buys on price alone. Nobody measures the damage — or the saving
oblem	45 sec	"2 million small contractors. Zero tools. No English. No computer. Just a phone and WhatsApp."
ve Demo	2 min	Follow the demo script exactly. Do not improvise.
ch Stack	30 sec	"Gemini Vision + Firebase + Google TTS. Scalable from day one. Built in 4 hours on $5 of credits."
siness Model	30 sec	"Free for contractors. Rs. 50 per supplier lead. Rs. 325,000/month by Year 1."
k	15 sec	"We need Google Cloud credits and a mentor to reach 1,000 contractors in Lahore by Q3."

One-Line Pitch — Memorise This

"Hum Pakistan ke 2 million thekedaron ka AI munshi hain — point karo, scan karo, smarter khareedaro."

"We are the AI assistant for Pakistan's 2 million contractors — point, scan, buy smarter."

Final Integration Checklist

Complete this together in Hour 3–4. All four members present.

#	Checkpoint	Who Verifies	Done?
1	Backend server running: http://localhost:8000/health returns {"status":"running"}	M3	■
2	Frontend loads at http://localhost:5173 with green EcoScan UI	M2	■
3	Scan a real brick photo → result appears with material name, carbon, cost, alternative	All	■
4	Urdu audio plays automatically after scan (no button press needed)	All	■
5	WhatsApp button opens with pre-filled supplier message	M4	■
6	Scan is logged to Firestore: Firebase Console → Firestore → scans collection shows new entry	M3	■
7	Live Firebase Hosting URL works on a phone browser (not just laptop)	M1	■
8	Demo script rehearsed at least 2 times with real photos	M4	■
9	Backup demo video recorded and saved on phone	M4	■
10	All API keys removed from any files that will be shown publicly on screen	M1	■

Why You Win

✓	Only team using Gemini Vision live on stage — real AI, not a mock or a screenshot
✓	Urdu voice output — no other team will have this. Judges will remember it.
✓	WhatsApp integration — shows you understand how Pakistan actually works
✓	Pakistan-specific problem — judges respect local relevance and real users
✓	Google stack used properly — Gemini + Firebase + TTS fits the competition brief exactly
✓	Working prototype + polished pitch — most teams will have slides only
✓	Realistic business model — Rs. 325,000/month Year 1. Believable and specific.

Good luck. You've got this.

ITU Startup Competition 2026 — Team EcoScan PK